Movie Revenue Prediction

Eric Wu, Matteo Martone

Introduction

Background

The film industry faces big financial risks, with studios investing millions in production and marketing without a guarantee of success. High stakes and unpredictable market conditions mean that having a reliable way of predicting movie revenues is crucial. When factors like changing viewer tastes and economic challenges come into play, older methods often fail to capture the ever-changing nature of the industry, which can lead to expensive mistakes.

The move toward more data-driven methods shows the need to cut these risks while making planning better. Using historical data and modern tools helps turn complex market signals into practical insights. This change not only aims to improve revenue predictions but also to give a clearer picture of the factors behind financial success, ultimately supporting better planning and investment decisions.

Goal and Objective

Our goal is to develop a robust, data-driven model that predicts revenue over a half-year window for future movie releases with near state-of-the-art accuracy. We will achieve this by evaluating the influence of pre-release metadata on movie revenues, adjusting for dollar inflation using the Consumer Price Index (CPI) to standardize revenue figures across different time periods, and ensuring a proper time-ordered approach where only past data is used to predict future events, thereby preventing data leakage. Additionally, we aim to identify key indicators from the metadata that have a significant impact on the financial success of a movie.

In addition to building our predictive model, we plan to deploy it in a user-friendly manner. To do so, we will set up a SQL database connection to support our FASTAPI application, design the frontend using CSS, HTML, and JavaScript, and deploy the predictive model as an interactive web feature.

Prior Work

Method

Data Collection and Feature Engineering

Starting with data collection, our group chose to use the official IMDB non-commercial datasets available at https://developer.imdb.com/non-commercial-datasets/. This source is updated daily and is highly compatible with scraping tools. From these official datasets, we obtained many primary attributes such as title, directors, writers, actors, IMDB rating, number of votes, release date, runtime, and genres.

In addition to the IMDB dataset, we also found a secondary dataset on GitHub that includes supplementary variables such as trailer views, trailer likes, revenue, budget, keywords, and more. By incorporating this

dataset, we avoided the ongoing hassle of sourcing box office data manually. However, since this dataset is static, we lose the ability to refresh it using scraper tools. To implement a complete Extract, Transform, Load (ETL) pipeline, we would need to identify a legal and up-to-date source for scraping box office numbers and use the YouTube API—combined with natural language processing (NLP) techniques—to extract trailer-related metrics such as views and likes.

Additionally, we attempted to scrape actor, director, and writer "billboards" for feature engineering purposes. We ensured that the sources used were recently updated and broadly representative. For directors and writers, we used an IMDb list created by a user, which was most recently updated on April 15, 2025, and has received 98,000 views. For actors, we used The Numbers billboard to collect actor information and corresponding star scores. We acknowledge that these billboards—especially those created by users—may contain biases, and addressing this bias is a potential area for future improvement.

Moving on to data processing, we removed all movies with null values and still retained a relatively sufficient number of records for model training. We also eliminated duplicate entries by using a composite primary key consisting of the movie title and release date. After cleaning the data, we began feature engineering. Our encoding approaches for each categorical variable are as follows:

- **Director and Writer**: We counted the total number of famous directors and writers associated with each movie, based on their appearance in our billboard lists.
- **Actor**: In addition to counting the number of famous actors, we also computed the cumulative star score for each movie.
- **Genre**: Since the genre field contains comma-separated strings, we extracted only the primary genre (i.e., the first listed) and created dummy variables based on these values.
- **Release Date**: We created two binary variables—is_holiday_release and is_competitive_month. The holiday variable flags releases near major holidays (e.g., Christmas, Thanksgiving, Halloween, Independence Day, Memorial Day, Labor Day, Valentine's Day, and Chinese New Year), while the competitive month variable flags historically high-grossing periods (May, June, July, November, and December).
- **Production Companies**: We created a binary variable, production_popular, to indicate whether a movie was produced by a major company. Our list includes: Warner Bros, Universal Pictures, Walt Disney, Columbia Pictures, 20th Century Fox, Paramount, Marvel Studios, New Line, DreamWorks, Lionsgate, MGM, Sony Pictures, Lucasfilm, Pixar, Legendary, A24, Focus Features, Amblin, Working Title, StudioCanal, and Castle Rock. In the future, we may use string similarity measures (e.g., cosine similarity) to better capture variations in company names.
- **Languages**: We selected the top five most common languages and created dummy variables for each. All other languages were grouped into an "Other Languages" category. In the future, we may consider combining these into a single variable to reduce multicollinearity.
- **Keywords**: We used the pretrained BERT model 'all-mpnet-base-v2' to embed each keyword into high-dimensional vectors. We then applied Principal Component Analysis (PCA) to reduce the dimensionality, selecting the first principal component. We ensured that the first eigenvalue explained at least 40% of the variance. Using a model like BERT helps preserve the semantic meaning of keywords.

By incorporating these features, we greatly expanded our pool of predictors and enhanced the expressiveness of our dataset for model training.

Star Score

The Star Score feature aims to quantify the impact of cast, directors, and writers on a movie's potential box-office performance by leveraging external "fame" data. In this section, we outline a general approach

to computing and integrating Star Score into our modeling pipeline. As more detailed information becomes available—such as finalized name lists, precise scoring metrics from thenumbers.com, and actor metadata—this section will be updated and refined.

Intuition: High-profile talent often drives audience interest and marketing reach, potentially boosting revenue.

Objective: Create numerical features that capture both the presence of recognized industry figures (directors, writers) and the cumulative prominence of lead actors.

Linear Modeling

In this section we summarize the linear-based approaches you have implemented, report their performance, examine diagnostic plots, and identify concrete next steps to push R-Squared above 0.92 and RMSE below $7M.

Ordinary Least Squares (OLS) Regression Ordinary Least Squares (OLS) is the foundational linear regression technique. It fits a hyperplane that minimizes the sum of squared residuals (differences between observed and predicted values). OLS is prized for its simplicity and interpretability—each coefficient represents the expected change in the target (revenue) for a one-unit change in the predictor, holding others constant. However, OLS assumes:

- Linearity between features and target
- Homoskedasticity (constant variance of residuals)
- Normality of residuals
- No multicollinearity among predictors

Violations (e.g., heteroskedasticity, collinearity) can lead to biased estimates or inflated variances.

Target: Log-transformed revenue, back-transformed for RMSE/R-Squared computation

Validation: 5-fold KFold (chronological, no shuffle), then hold-out test on the newest 20% of films

Cross-validaddted RMSE: 19.43 Millions Cross-validated R-Squared: 0.79

RMSE: 9.01 Million R-Squared: 0.86

Diagnostic: Residuals vs Fitted shows clear heteroskedasticity (variance increases with fitted revenue).

Q-Q plot exhibits heavy right tail—extreme overestimation for blockbusters.

Histogram of residuals is sharply peaked with heavy tails.

No formal outlier removal or robust weighting was applied.

Elastic Net Regression Elastic Net combines Lasso (L1) and Ridge (L2) penalties to control model complexity and handle multicollinearity. The L1 penalty can shrink some coefficients exactly to zero (feature selection), while L2 distributes shrinkage across correlated predictors. Elastic Net is well-suited when predictors are correlated—unlike Lasso, which arbitrarily picks one, Elastic Net keeps groups of correlated features together.

Results:

- RMSE: 8.09 Millions

- R-Squared: 0.89

Regularization reduced overfitting slightly (RMSE 0.9M down, R-Squared 0.03 up).

Coefficients remain interpretable; none were driven exactly to zero given high l1_ratio.

Takeaway: Elastic Net improves predictive accuracy by reducing overfitting and stabilizing coefficient estimates in the presence of moderate collinearity (e.g., between marketing metrics). It balances interpretability (non-zero coefficients) with regularization.

Generalized Additive Model (GAM) Generalized Additive Models (GAMs) extend linear models by fitting smooth functions (splines) to each predictor, allowing for flexible, nonlinear relationships while maintaining an additive structure. Each term SJXJ estimated via penalized splines, and overall interpretability is preserved through partial dependency plots, which show how the target responds to each feature holding others constant.

Performance

Test RMSE = $7.33M (best overall)

Test R-Squared = 0.91 (closest to stretch goal)

MAE: 3.27 M

Partial Dependence:

- Trailer Likes (log): Roughly linear positive effect—more trailer engagement reliably boosts revenue expectations.
- Budget (log): Nonlinear pattern—diminishing returns at mid-range budgets, then a sharp rise for very large budgets (blockbusters).
- Budget×Actor Interaction: A concave relationship—films with combined high budgets and star casts earn disproportionately more until a plateau.
- Release Year: Gradual decline for the most recent releases, possibly reflecting market saturation or pandemic effects.
- Takeaway: GAM captures nuanced, nonlinear effects that OLS and Elastic Net cannot. The partial plots guide data-driven decisions—e.g., optimal budget ranges and year effects.4.5 Limitations & Potential Next Steps

Heteroskedasticity

- Residuals fan out at higher fitted values. Next: Implement weighted least squares or use robust standard errors (HC3) in OLS.

Summary and Next Steps Key Insights:

- Interpretability vs. Flexibility: OLS and Elastic Net yield straightforward coefficients, while GAM uncovers nonlinear patterns.
- Heteroskedasticity remains in all linear models; addressing it could reduce RMSE further.
- Outliers (blockbusters) drive error—targeted down-weighting or robust loss functions may help.

Next Steps

- Heteroskedasticity: Implement Weighted Least Squares or robust standard errors for OLS/Elastic Net.
- Outlier Management: Identify high-leverage points via Cook's distance and consider trimming or Winsorizing.
- GAM Tuning: Optimize spline degrees (number of knots) to further improve R-Squared.
- Interaction Terms: Explore additional interactions (e.g., trailer engagement × release month) in regularized models.

Goal: Push Test RMSE below $7M and Test R-Squared above 0.92

Nonlinear Modeling

Result

Data

Model Result

SQL Connection

We successfully implemented a MySQL database, which allows our data to benefit from its Atomicity, Consistency, Isolation, and Durability (ACID) properties. These properties help ensure that each transaction is reliable, errors don't leave the data in an inconsistent state, concurrent processes don't conflict, and changes are permanently saved once committed.

Beyond ACID, the database also improves performance and scalability. With a well-defined schema, data insertion follows strict rules, which helps maintain data integrity. We can easily export tables as CSV files for future use or offline analysis.

The database also integrates well with the website: the upload function checks for duplicate records, and we've built API endpoints for deleting and updating data. This makes it easy to manage content in real time without compromising accuracy or stability.

Website Navigation

By having a database connection, we implemented website functionality using the FastAPI framework. The FastAPI Python module allows us to quickly set up a basic functioning website that supports Create, Read, Update, and Delete (CRUD) endpoints.

To further enhance website functionality, we also experimented with CSS styling and JavaScript functions to toggle some of the API endpoints and other features, such as uploading new data and making predictions. We hope this will enhance the user experience with the prediction model.

Tabelau Dashboard

Discussion

Challenges and Limitation

Next Step

Conclusion

Reference

- Testing